

Redes Neuronales

U3 Redes Neuronales Multicapa

Héctor Maravillo

Section 1

The backpropagation algorithm

The backpropagation algorithm

Let us assume that the **neural network** consist of a fixed number of L **layers** of neurons, with $k_0 = I$ nodes in the input layer and k_r **neurons** in the r th layer, for $r = 1, 2, \dots, L$.

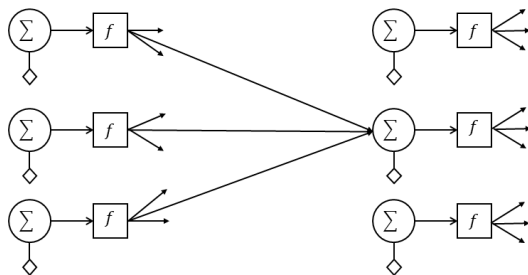
Suppose that all the neurons employ the same **sigmoid activation function**

$$f(x) = \frac{1}{1 + \exp(-ax)}$$

where a is a **slope parameter**.

The backpropagation algorithm

Multilayer Neural Network



- L Number of layers
- $k_0 = l$ Number of neurons in the input layer
- k_r Number of neurons in the layer $r = 1, \dots, L$
- N Number of observations

The backpropagation algorithm

We assume that N **training pairs** are available

$$(\mathbf{x}(i), \mathbf{y}(i)), \quad i = 1, 2, \dots, N$$

For convenience, we use i in parenthesis and not as an index, to refer the instance.

- Because we have now assumed k_L **output neurons**, the output is no longer a scalar, the output is not a scalar but a k_L -dimensional vector (for instance, think k_L classes):

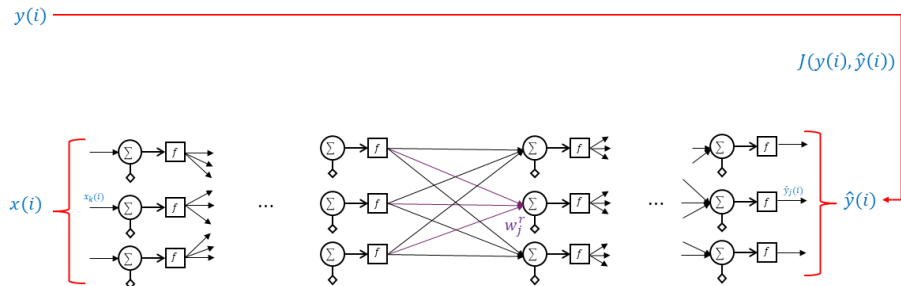
$$\mathbf{y}(i) = [y_1(i), \dots, y_{k_L}(i)]^T$$

- The **input vectors** are k_0 -dimensional vectors

$$\mathbf{x}(i) = [x_1(i), \dots, x_{k_0}(i)]^T$$

During training, when vector $\mathbf{x}(i)$ is applied to the input, the **output of the network** will be $\hat{\mathbf{y}}(i)$, which is different from the **desired value** $\mathbf{y}(i)$.

The backpropagation algorithm



The backpropagation algorithm

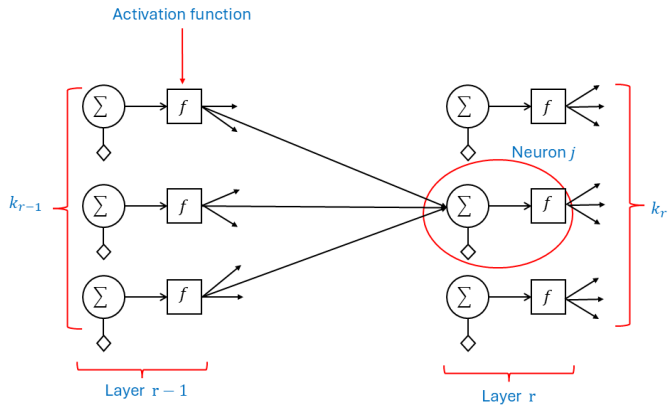
The **synaptic weights** are computed such that an appropriate (for each problem) **cost function** J is **minimized**, which is dependent on the values $\mathbf{y}(i)$ and $\hat{\mathbf{y}}(i) = \phi(\mathbf{x}(i), \mathbf{w})$, $i = 1, 2, \dots, N$. In other words, we are trying to solve the **unconstrained optimization problem**:

$$\min J(\mathbf{y}, \hat{\mathbf{y}})$$

The cost function J depends, through $\hat{\mathbf{y}}(i)$ **on the weights** and that this is a **nonlinear dependence**, due to the nature of the network itself (**activation function**).

Thus, minimization of the cost function can be achieved via iterative techniques. The most used approach is **gradient descent scheme**.

The backpropagation algorithm



The backpropagation algorithm

Let $\mathbf{w}_j^r$ be the **weight vector** (including the threshold) of the j th **neuron** in the r th **layer**, which is a vector of dimension $k_{r-1} + 1$ and is defined as

$$\mathbf{w}_j^r = [w_{j0}^r, w_{j1}^r, \dots, w_{jk_{r-1}}^r]^T$$

The basic iteration step will be of the form

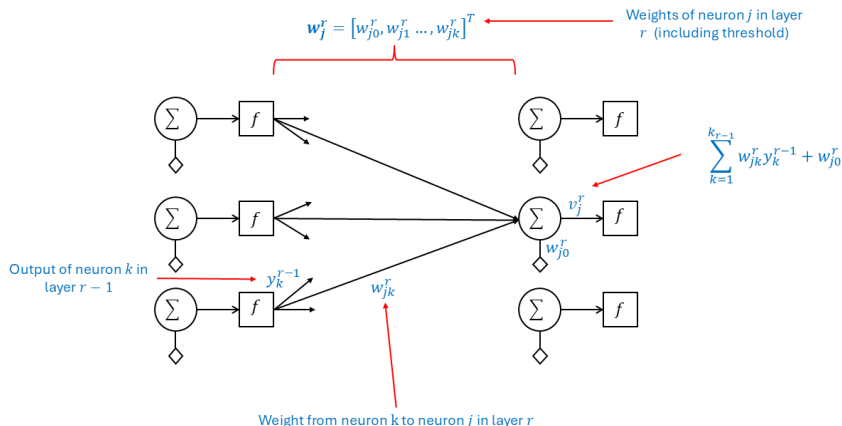
$$\mathbf{w}_j^r(\text{new}) = \mathbf{w}_j^r(\text{old}) + \Delta \mathbf{w}_j^r$$

with

$$\Delta \mathbf{w}_j^r = -\mu \frac{\partial J}{\partial \mathbf{w}_j^r}$$

where $\mathbf{w}_j^r(\text{old})$ is the **current estimate** of the unknown weights and $\Delta \mathbf{w}_j^r$ the corresponding **correction** to obtain the next estimate $\mathbf{w}_j^r(\text{new})$, with μ a suitable **learning rate**.

The backpropagation algorithm



In Figure, y_j^r is the weighted summation of the inputs to the j th neuron of the r th layer and y_k^{r-1} the corresponding output after the activation function.

The backpropagation algorithm

We will focus our attention on **cost functions** of the form

$$J = \sum_{i=1}^N \varepsilon(i)$$

where ε is an appropriately defined function depending on $\hat{\mathbf{y}}(i)$ and $\mathbf{y}(i)$, $i = 1, 2, \dots, N$.

In other words, the function J is expressed as a sum of the N values that function ε takes for each of the training pairs $(\mathbf{x}(i), \mathbf{y}(i))$ (because $\mathbf{y}(i)$ is a function of the instance $\mathbf{x}(i)$ and the weights).

For example, we can choose $\varepsilon(i)$ as the **sum of squared errors** in the output neurons

$$\varepsilon(i) = \frac{1}{2} \sum_{m=1}^{k_L} e_m^2(i) = \frac{1}{2} \sum_{m=1}^{k_L} (\mathbf{y}_m(i) - \hat{\mathbf{y}}_m(i))^2, \quad i = 1, 2, \dots, N$$

Computation of the gradients

For the computation of the correction term

$$\Delta \mathbf{w}_j^r = -\mu \frac{\partial J}{\partial \mathbf{w}_j^r}$$

The **gradient** of the **cost function** with respect to the weights is required and, consequently, the evaluation of

$$\frac{\partial \varepsilon(i)}{\partial \mathbf{w}_j^r}$$

Computation of the gradients

- Let $y_k^{r-1}(i)$ be the output of the k th neuron, $k = 1, 2, \dots, k_{r-1}$, in the $(r-1)$ th layer for the i th training pair
- Let w_{jk}^r be the current estimate of the corresponding weight leading to the j th neuron in the r th layer, with $j = 1, 2, \dots, k_r$.

Thus the argument of the activation function $f(\cdot)$ of the latter neuron will be

$$v_j^r(i) = \sum_{k=1}^{k_{r-1}} w_{jk}^r y_k^{r-1}(i) + w_{j0}^r \quad (1)$$

$$= \sum_{k=0}^{k_{r-1}} w_{jk}^r y_k^{r-1}(i) \quad (2)$$

where by definition $y_0^r(i) \equiv +1, \forall r, i$; so as to include the threshold in the weights.

Computation of the gradients

For the output layer, we have $r = L$,

$$y_k^r(i) = y_k^L(i) = \hat{y}_k(i), \quad k = 1, \dots, k_L$$

that is, the outputs of the neural network, and for $r = 1$

$$y_k^{r-1}(i) = y_k^0(i) = x_k(i), \quad k = 1, \dots, k_0$$

that is, the network inputs.

Computation of the gradients

The dependence of $\varepsilon(i)$ on $\mathbf{w}_j^r$ passes through $v_j^r(i)$. By the **chain rule** in differentiation, we have

$$\frac{\partial \varepsilon(i)}{\partial \mathbf{w}_j^r} = \frac{\partial \varepsilon(i)}{\partial v_j^r(i)} \frac{\partial v_j^r(i)}{\partial \mathbf{w}_j^r}$$

From

$$v_j^r(i) = \sum_{k=0}^{k_{r-1}} w_{jk}^r y_k^{r-1}(i)$$

we obtain

$$\frac{\partial v_j^r(i)}{\partial \mathbf{w}_j^r} \equiv \begin{bmatrix} \frac{\partial}{\partial w_{j0}^r} v_j^r(i) \\ \frac{\partial}{\partial w_{j1}^r} v_j^r(i) \\ \vdots \\ \frac{\partial}{\partial w_{jk_{r-1}}^r} v_j^r(i) \end{bmatrix} = \begin{bmatrix} 1 \\ y_1^{r-1}(i) \\ \vdots \\ y_{k_{r-1}}^{r-1}(i) \end{bmatrix} = \mathbf{y}^{r-1}(i)$$

Computation of the gradients

Let us define

$$\frac{\partial \varepsilon(i)}{\partial v_j^r(i)} \equiv \delta_j^r(i)$$

then the correction term $\Delta \mathbf{w}_j^r = -\mu \frac{\partial J}{\partial \mathbf{w}_j^r}$ becomes

$$\Delta \mathbf{w}_j^r = -\mu \sum_{i=1}^N \delta_j^r(i) \mathbf{y}^{r-1}(i)$$

This relation is general for **any differentiable** cost function

$$J = \sum_{i=1}^N \varepsilon(i)$$

Computation of $\delta_j^r(i)$ for least squares function

We will compute $\delta_j^r(i)$ for the special case of least squares

$$\varepsilon(i) = \frac{1}{2} \sum_{m=1}^{k_L} (y_m(i)) - \hat{y}_m(i))^2, \quad i = 1, 2, \dots, N$$

The procedure is similar for alternative cost function choices.

The computations start from $r = L$ and **propagate backward** for $r = L - 1, L - 2, \dots, 1$. This is why the algorithm is known as **the backpropagation algorithm**.

Computation of $\delta_j^r(i)$ for least squares function

- 1. $r = L$

$$\delta_j^L(i) = \frac{\partial \varepsilon(i)}{\partial v_j^L(i)}$$

$$\varepsilon(i) \equiv \frac{1}{2} \sum_{m=1}^{k_L} e_m^2(i) \equiv \frac{1}{2} \sum_{m=1}^{k_L} \left(f(v_m^L(i)) - y_m(i) \right)^2$$

hence

$$\delta_j^L(i) = e_j(i) f'(v_j^L(i)) \quad (3)$$

$$= \left(f(v_j^L(i)) - y_j(i) \right) f'(v_j^L(i)) \quad (4)$$

where f' is the derivative of $f(\cdot)$.¹

¹In the last layer, the dependence of $\varepsilon(i)$ on $v_j^L(i)$ is explicit, and the computation of the derivative is straightforward. This is not true, however, for the hidden layers, where the computations of the derivatives need more elaboration.

Computation of $\delta_j^r(i)$ for least squares function

2. $r < L$

Due to the successive dependence among the layers, the value of $v_j^{r-1}(i)$ influences all $v_k^r(i)$, $k = 1, 2, \dots, k_r$, of the next layer.

Employing the **chain rule** in differentiation once more, we obtain

$$\frac{\partial \varepsilon(i)}{\partial v_j^{r-1}(i)} = \sum_{k=1}^{k_r} \frac{\partial \varepsilon(i)}{\partial v_k^r(i)} \frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)}$$

And from the definition

$$\delta_j^r(i) \equiv \frac{\partial \varepsilon(i)}{\partial v_j^r(i)}$$

we obtain

$$\delta_j^{r-1}(i) = \sum_{k=1}^{k_r} \delta_k^r(i) \frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} \quad (5)$$

Computation of $\delta_j^r(i)$ for least squares function

But

$$\frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} = \frac{\partial \left[\sum_{m=0}^{k_{r-1}} w_{km}^r y_m^{r-1}(i) \right]}{\partial v_j^{r-1}(i)}$$

with

$$y_m^{r-1}(i) = f(v_m^{r-1}(i))$$

Hence

$$\frac{\partial v_k^r(i)}{\partial v_j^{r-1}(i)} = w_{kj}^r f' \left(v_j^{r-1}(i) \right) \quad (6)$$

Computation of $\delta_j^r(i)$ for least squares function

From (5) y (6) the following results:

$$\delta_j^{r-1}(i) = \sum_{k=1}^{k_r} \left[\delta_k^r(i) w_{kj}^r f' \left(v_j^{r-1}(i) \right) \right] \quad (7)$$

$$= \left[\sum_{k=1}^{k_r} \delta_k^r(i) w_{kj}^r \right] f' \left(v_j^{r-1}(i) \right) \quad (8)$$

$$= e_j^{r-1}(i) f' \left(v_j^{r-1}(i) \right) \quad (9)$$

Relations (5) and (8) constitute the iterations leading to the computation of $\delta_j^r(i)$, $r = 1, 2, \dots, L$, $j = 1, 2, \dots, k_r$

The Backpropagation Algorithm

- 1. **Initialization:**

Initialize all the weights with **small random values** from a pseudorandom sequence generator.

The Backpropagation Algorithm

2. Forward computations:

For each of the training feature vectors $x(i)$, $i = 1, 2, \dots, N$, compute all the v_j^r , $y_j^r(i) = f(v_j^r(i))$, $j = 1, 2, \dots, k_r$, $r = 1, 2, \dots, L$ from:

$$v_j^r(i) = \sum_{k=0}^{k_{r-1}} w_{jk}^r y_k^{r-1}(i)$$

Compute the cost function for the current estimate of weights from

$$\begin{aligned} J &= \sum_{i=1}^n \varepsilon(i) \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{m=1}^{k_L} \left(f(v_m^L(i)) - y_m(i) \right)^2 \end{aligned}$$

The Backpropagation Algorithm

3. Backward computations:

For each $i = 1, 2, \dots, N$ and $j = 1, 2, \dots, k_L$ compute

$$\begin{aligned}\delta_j^L(i) &= e_j(i) f'(v_j^L(i)) \\ &= \left(f(v_j^L(i)) - y_j(i) \right) f'(v_j^L(i))\end{aligned}$$

and in the sequel compute

$$\begin{aligned}\delta_j^{r-1}(i) &= e_j^{r-1}(i) f' \left(v_j^{r-1}(i) \right) \\ &= \left[\sum_{k=1}^{k_r} \delta_k^r(i) w_{kj}^r \right] f' \left(v_j^{r-1}(i) \right)\end{aligned}$$

for $r = L, L-1, \dots, 2$ and $j = 1, 2, \dots, k_r$.

The Backpropagation Algorithm

● 4. Update the weights:

For $r = 1, 2, \dots, L$, and $j = 1, 2, \dots, k_r$

$$\mathbf{w}_j^r(\text{new}) = \mathbf{w}_j^r(\text{old}) + \Delta \mathbf{w}_j^r \quad (10)$$

$$\Delta \mathbf{w}_j^r = -\mu \sum_{i=1}^N \delta_j^r(i) \mathbf{y}^{r-1}(i) \quad (11)$$

Stop criterion

A number of criteria have been suggested for terminating the iterations.

For instance,

- When the **cost function** J becomes **smaller** than a **certain threshold**.
- When its **gradient** with respect to the weights becomes **small**.

Learning ratio

As with all the algorithms that spring from the **gradient descent method**, the **convergence speed** of the backpropagation scheme depends on the value of the **learning constant** μ .

Its value must be sufficiently small to guarantee convergence but not too small, because the convergence speed becomes very slow.

The best choice of μ depends very much on the problem and the cost function shape in the weight space.

Classification

Once **training** of the network has been achieved, the values to which the synapses and thresholds have converged are frozen, and the network is ready for **classification**.

This is a much easier task than training. An unknown feature vector is presented in the input and is classified in the class that is indicated by the output of the network.

The computations performed by the neurons are of the multiply-add type followed by a nonlinearity

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009